ECM3446/ECMM461:
High Performance Computing

UNIVERSITY OF EXETER

Variable scoping

Unit 2.6

Variable scoping

- In the examples in the previous unit different threads accessed the same copies of arrays **`x`** and **`y`** (shared)

- The loop index **`i`** was different for each thread (private)

- The correct functioning of an OpenMP loop requires correct variable scoping

- In this case the default scoping rules did the right thing

```c
#include <stdio.h>
#include <math.h>
#include <omp.h>

int main(){

  const int N=12;
  float x[N],y[N];

#pragma omp parallel for
  for (int i=0; i<N; i++){
    x[i]= (float) (i+1);
    y[i]= log(x[i]);
  }

  for (int i=0; i<N; i++){
    printf("%g %g\n", x[i], y[i]);
  }

  return 0;
}
```

shared

private

Declaring variable scope

- Explicitly declaring variable scope makes the code much easier to understand and more likely to be correct

- We can specify the default scope by adding a clause to the directive which starts the parallel region:

  - `default (shared)`

  - `default (none)`

- The default clause can be followed by a list of private and/or shared variables

Don’t need to declare `N` (constant) or `i` (declared in parallel region)

```
#include <stdio.h>
#include <math.h>
#include <omp.h>

int main(){

  const int N=12;
  float x[N],y[N];

#pragma omp parallel for default (none) shared(x, y)
  for (int i=0; i<N; i++){
    x[i]= (float) (i+1);
    y[i]= log(x[i]);
  }

  for (int i=0; i<N; i++){
    printf("%g %g\n", x[i], y[i]);
  }

  return 0;
}
```

- Now we will try calculating the sum of integers from 1 to N in parallel

- We know the answer should be:

$$\sum_{i=1}^{N} i = \frac{1}{2} N \left(N + 1 \right)$$

```c
#include <stdio.h>

const int N=100;
int i;
int sum;

int main(){

   sum=0;
#pragma omp parallel for default(none) shared(sum) private(i)
   for (i=1; i<N+1; i++){
      sum += i;
   }

   printf("Sum= %d\n", sum);
   return 0;

}
```

Older style C where `i` is declared outside the loop iterator

Data races

- This is an example of a data race - a bug specific to parallel programming

- Multiple threads may try to update the variable **`sum`** at the same time

- Updates to the **`sum`** variable may be corrupted

- We need each thread to have a private copy of **`sum`** which is summed at the end of the parallel loop

Reduction variables

- **`reduction`**: combine results from multiple threads into a single value using a reduction operator
- `#pragma omp reduction(operator : list)`
- For our example we need:

```
#pragma omp for reduction(+ : x)
```

```c
#include <stdio.h>

const int N=100;
int i;
int sum;

int main(){

   sum=0;
#pragma omp parallel for default(none) reduction(+ : sum) private(i)
   for (i=1; i<N+1; i++){
     sum += i;
   }

   printf("Sum= %d\n", sum);
   return 0;

}
```

Make **`sum`** a reduction variable

Reduction operators: C

Reduction operator	Operation	Initial value
+	Addition	0
-	Subtraction	0
*	Multiplication	1
&	bitwise and	~0 (all bits on)
|	bitwise or	0
^	bitwise xor	0
&&	logical and	1
||	logical or	0
max	maximum value	Largest representable number for reduction variable type
min	minimum value	Largest representable number for reduction variable type

Unit summary

- Correctly scoping variables is vital for correct operation of an OpenMP loop

- If multiple OpenMP threads attempt to update the same variable simultaneously updates may be corrupted (data race)

- We saw how to scope variables with **`private`**, **`shared`** and **`reduction`** clauses

- There are other ways to scope variables in OpenMP but these clauses are frequently used in practice